

開発環境を整える 〜プログラミングの「キッチン」を準備しよう〜

悩み (Problem)

プログラミングを始めようとしたとき、Node.jsやVS Codeをインストールし、`npm run dev`のようなコマンドを実行すると、エラーメッセージが表示され、友達のPCでは動くのに、自分のPCではなぜ動かないのか疑問に思うかもしれ

解決策 (Solution)

このチュートリアルでは、Taskfileを使って開発環境を簡単にセットアップし、複雑なコマンドを覚える必要はありません。

最終的には、アプリを起動し、チェックを実行し、どのファイルが何をするか

Kitchen メタファーで理解する (Kitchen Metaphor Table)

プログラミング開発も、キッチンでの調理と同じです。
必要な道具と材料、そしてレシピが揃って初めて美味しい料理が作れるように

キッチン	技術	役割
コンロ	Node.js	プログラムを「調理」する
キッチンセット	Docker	全ての道具を箱に詰めて持ち運ぶ
冷蔵庫	PostgreSQL	食材（データ）を保存
レシピカード	Taskfile	複雑な手順を1ステップに
食材リスト	package.json	必要な材料と分量を記載
調理器具	npm (パッケージ)	コード作成に必要な道具

Taskfileの使い方 (How to Use Taskfile)

Taskfile (タスクファイル) は、複雑なコマンドを1つの言葉で実行できるTaskfile.yamlというファイルに、実行したい処理を記述しておきます。

// filepath: taskfile.yaml version: "3"

```
tasks: init: desc: "Setup project" cmds: - docker compose down --volumes - npm
ci --legacy-peer-deps - npx prisma migrate deploy - npm run seed dev: desc:
"Start dev server" cmds: - docker compose up -d && npm run dev format: desc:
"Format code" cmds: - prettier --write . lint: desc: "Lint code" cmds: - next lint
check: desc: "Type check" cmds: - tsc
```

確認ポイント:

- `version: "3"`はTaskfileのバージョンを示します。
- `tasks:`以下に、実行できるタスクが定義されています。
- `init`タスクは、プロジェクトの初期設定を行います。
- `dev`タスクは、開発サーバーを起動します。
- `format`タスクは、コードを自動整形します。
- `lint`タスクは、コードのスタイルチェックを行います。
- `check`タスクは、TypeScriptの型チェックを行います。

このように、複雑なコマンドが`task`コマンド1つにまとまっているため、
利用する主なコマンドは、`task init`、`task dev`、`task seed`で

コマンド	説明	実行タイミング
task init	Dockerとnpmを一括でセットアップします。	プロジェクトの初期設定時
task dev	開発サーバーを起動します。	開発作業を開始するとき
task seed	テストデータをデータベースに投入します。	データベースの初期化時
task format	コードのフォーマットを自動で行います。	コードの編集後
task lint	コードの品質チェックを行います。	コードの品質を確認するとき
task check	TypeScriptの型チェックを行います。	型エラーを確認するとき

セットアップステップバイステップ (Setup Steps)

Step 1: 前提条件の確認

まず、必要なツールがインストールされているか確認します。
ターミナル（コマンドプロンプト）を開き、以下のコマンドを実行してくだ

```
$ node -v v24.11.1
```

```
$ npm -v 10.9.0
```

```
$ docker --version Docker version 27.0.0
```

確認ポイント:

- `node -v`はNode.jsのバージョンを表示します。`v24.11.1`のように
- `npm -v`はnpmのバージョンを表示します。`10.9.0`のように表示され
- `docker --version`はDockerのバージョンを表示します。`Docker

もしバージョンが異なる場合でも、基本的には問題ありません。

ただし、Node.jsのバージョンが古すぎる場合は、最新版にアップデートす

Step 2: `task init` コマンドの実行

次に、`task init`コマンドを実行して、プロジェクトを初期化します。

```
$ task init task: [init] docker compose down --volumes task: [init] npm ci --
```

```
legacy-peer-deps task: [init] npx prisma migrate deploy task: [init] npm run seed
```

```
All setup complete!
```

確認ポイント:

- `docker compose down --volumes`は、以前のコンテナを停止し、戻
- `npm ci --legacy-peer-deps`は、package-lock.jsonに基づいて
- `npx prisma migrate deploy`は、データベースのスキーマを最新の
- `npm run seed`は、データベースに初期データを投入します。
- 最後に「All setup complete!」と表示されたら成功です。

Step 3: `task dev` で動作確認

`task dev`コマンドを実行して、開発サーバーを起動します。

```
$ task dev task: [dev] docker compose up -d && npm run dev Waiting for healthcheck... Next.js 15.1.6 (starting) Local: http://localhost:3000
```

確認ポイント:

- `docker compose up -d && npm run dev`は、Dockerコンテナを起動
- 「Waiting for healthcheck...」と表示された後、「<http://localhost:3000>
- ブラウザで<http://localhost:3000>を開くと、Next.jsの初期画面が表示

動作確認 (Verification)

Docker Desktopでの確認

Docker Desktopを開くと、2つのコンテナ (taskapp-postgres と taskapp-nextjs) が起動していることが確認できます。これは、PostgreSQLデータベースとバックエンドサーバーが正常に動作していることを示しています。

ブラウザでの確認

ブラウザで`<http://localhost:3000>`にアクセスすると、Next.jsの初期画面が表示されます。もし画面が表示されない場合は、ターミナルでエラーが発生していないか確認してください。

データベースの確認

データベースが正常に起動していることを確認するために、`task db:studio`を実行してPrisma Studioを開きます。

```
$ task db:studio task: [db:studio] npx prisma studio Prisma Studio is running at http://localhost:5555
```

ブラウザでhttp://localhost:5555を開くと、データベースの内容を確認

Mermaid Diagram: Docker Compose Architecture (Technical)

```
``mermaid
graph LR
  A[User] -->|localhost:3000| B[Next.js Frontend]
  B -->|API| C[Node.js Backend]
  C -->|Query| D[PostgreSQL]
  E[Taskfile] -->|docker compose| F["Docker Compose<br/>"]
```

確認ポイント:

- User (ユーザー) は、ブラウザを通じてNext.js Frontend (フロントエンド) にアクセスします。
- Next.js Frontend (フロントエンド) は、APIを通じてNode.js Backend (バックエンド) にリクエストを送信します。
- Node.js Backend (バックエンド) は、PostgreSQL (データベース) にクエリを送信し、データを取得します。
- Taskfileは、Docker Composeを操作して、これらのサービスをまとめて管理します。

トラブルシューティング (Troubleshooting)

問題	解決策
<code>task init</code> がエラーで停止する	Dockerが起動しているか確認してください。
<code>task dev</code> が起動しない	ポート3000が他のプロセスで使用されていないか確認してください。
ブラウザに何も表示されない	ターミナルにエラーメッセージが表示されていないか確認してください。
データベースに接続できない	環境変数 <code>DATABASE_URL</code> が正しく設定されているか確認してください。

まとめ（Summary）

Taskfileを使えば、複雑なセットアップも1コマンドで完了します。コンロ、キッチンセット、冷蔵庫、レシピカード...すべてが揃って、初めて調理ができます。次のチャプターでは、Taskfileの仕組みを詳しく説明します。 ``